



Security Review For Lista DAO



Collaborative Audit Prepared For: **Lista DAO**
Lead Security Expert(s): **pkqs90**
vinica_boy
Date Audited: **May 7 - May 10, 2026**

Introduction

Lista DAO is a decentralized liquidity protocol on BNB Chain that combines liquid staking and a decentralized stablecoin (lisUSD). Users can stake BNB to receive slisBNB, a yield-bearing liquid staking token, and use a wide range of crypto assets as collateral to mint lisUSD, an over-collateralized, decentralized stablecoin. Lista also offers a lending market (Lista Lending) powered by Moolah, enabling efficient borrowing and lending across the BNB Chain ecosystem. The protocol is governed by the LISTA token and aims to become the liquidity hub of BNB Chain.

Scope

Repository: lista-dao/lista-new-contracts

Audited Commit: 7084d49bb14f567f407e720f88bca43744cc00e8

Final Commit: 24734ee60eb23d8c49f174f78e8b7557c012506c

Files:

- src/lisaster/AsterVault.sol
- src/lisaster/LisAsterDistributor.sol
- src/lisaster/LisAsterRewards.sol
- src/lisaster/LisAster.sol
- src/lisaster/LisAsterStaking.sol

Final Commit Hash

24734ee60eb23d8c49f174f78e8b7557c012506c

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	5

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: Malformed merkle roots can pay more rewards than the declared allocation [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-04-lista-dao-may-7th-2026/issues/6>

Vulnerability Detail

LisAsterDistributor uses `totalAllocated` to represent the cumulative rewards allocated by the live Merkle root. When BOT stages and accepts a root, the contract only checks that the declared `newTotalAllocated` is monotonic and does not exceed `totalNotified`.

```
function setPendingMerkleRoot(bytes32 root, uint256 newTotalAllocated) external  
    ↪ onlyRole(BOT) {  
    @> require(newTotalAllocated >= totalAllocated, "allocated decrease");  
    @> require(newTotalAllocated <= totalNotified, "exceeds notified");  
    ...  
}
```

However, claims are paid only from the Merkle leaf's `cumulativeAmount`. `_consume()` never checks that the paid amount fits within the remaining declared allocation.

```
payable_ = cumulativeAmount - already;  
claimed[account] = cumulativeAmount;  
totalClaimed += payable_;
```

A malformed root can declare `totalAllocated = 1 ether` while containing a valid leaf for 10 ether. The user can then claim 10 ether, making `totalClaimed` exceed `totalAllocated`.

Note if offchain component is always trusted and works as expected, this is not an issue.

Impact

`totalAllocated` no longer reliably bounds payouts. A bad accepted root can pay more rewards than declared, breaking allocation monitoring and the `totalClaimed <= totalAllocated` invariant.

Recommendation

In `_consume()`, enforce the allocation bound:

```
require(totalClaimed + payable_ <= totalAllocated, "exceeds allocated");
```

Issue L-2: Emergency withdrawals can make user claims and unstaking insolvent [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-04-lista-dao-may-7th-2026/issues/7>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

`LisAsterStaking` accounts user deposits through `balanceOf` and `totalSupply`, while holding the actual `lisAster` escrow. `LisAsterDistributor` accounts reward funding through `totalNotified` and `totalClaimed`.

Both contracts allow `MANAGER` to withdraw arbitrary tokens through `emergencyWithdraw()` without checking whether the withdrawn `lisAster` is surplus.

```
function emergencyWithdraw(address token, uint256 amount) external  
→ onlyRole(MANAGER) {  
    IERC20(token).safeTransfer(msg.sender, amount);  
}
```

If `MANAGER` withdraws accounted `lisAster`, the internal accounting remains unchanged while the backing tokens are removed.

Impact

Users may be unable to `un stake()` from `LisAsterStaking` or `claim()` from `LisAsterDistributor` because the contracts no longer hold enough `lisAster` to satisfy recorded obligations.

This also potentially breaks the invariant:

- `lisAster.balanceOf(Distributor) == totalNotified - totalClaimed`
- `LisAsterStaking.balanceOf(this) == totalSupply` (i.e. `lisAster.balanceOf(staking) == Σ user balances`)

Recommendation

Only allow `lisAster` emergency withdrawals for surplus above accounted obligations.

Issue L-3: Stakers can receive rewards for momentary stakes [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-04-lista-dao-may-7th-2026/issues/8>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

In `LisAsterStaking`, rewards are based on off-chain hourly reads of `balanceOf`. However, `take()` and `unstake()` allow users to enter and exit immediately.

```
function stake(uint256 amount) external override whenNotPaused nonReentrant {
    _stake(msg.sender, amount);
}

function unstake(uint256 amount) external override whenNotPaused nonReentrant {
    balanceOf[msg.sender] -= amount;
    totalSupply -= amount;
    IERC20(lisAster).safeTransfer(msg.sender, amount);
}
```

A user can stake shortly before the hourly snapshot and unstake immediately after, receiving rewards as if they were staked for the whole period.

Impact

Rewards can be misallocated to short-term stakers, diluting honest users who keep funds staked for the full period.

Recommendation

The offchain snapshot mechanism is a blackbox so it is not possible to perfectly grief this. So this is an informational issue. The best fix (but also may be overkill) would be use time-weighted balances, a minimum staking duration, or a cooldown before balances become eligible for reward snapshots.

Issue L-4: Emergency withdrawals can break staking solvency while unpaused [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-04-lista-dao-may-7th-2026/issues/9>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

In `LisAsterStaking`, `emergencyWithdraw()` lets `MANAGER` withdraw any token at any time. It does not require the contract to be paused (even though the code comments mentions it should be paused) and does not prevent withdrawing staked `lisAster` principal.

```
///      `balanceOf(this) >= totalSupply` invariant; pause first and reconcile
↪  by runbook.
function emergencyWithdraw(address token, uint256 amount) external
↪  onlyRole(MANAGER) {
    require(token != address(0), "zero token");
    require(amount > 0, "zero amount");
    IERC20(token).safeTransfer(msg.sender, amount);
}
```

If `lisAster` is withdrawn below `totalSupply`, the staking contract becomes insolvent while normal staking and unstaking may still be active.

Impact

A manager mistake or compromised manager key can break `lisAster.balanceOf(staking) >= totalSupply` invariant, causing user withdrawals to fail or leaving staked balances undercollateralized.

Recommendation

1. Require `whenPaused` for `emergencyWithdraw()`.
2. `lisAster` should only allow withdrawing surplus above `totalSupply`.

Issue L-5: changeWaitingPeriod() can shrink the veto window of an in-flight pending Merkle root [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-04-lista-dao-may-7th-2026/issues/10>

Summary

`acceptMerkleRoot()` reads the live `waitingPeriod` instead of the value at stage time, so lowering `waitingPeriod` while a root is pending shortens MANAGER's veto window for that staged root.

Vulnerability Detail

`setPendingMerkleRoot()` records `lastSetTime` but does not snapshot `waitingPeriod`. `acceptMerkleRoot` checks `block.timestamp >= lastSetTime + waitingPeriod` against the current value. If `DEFAULT_ADMIN` calls `changeWaitingPeriod` with a smaller value while a root is pending, the staged root becomes acceptable sooner than the period that was in effect at stage time.

The 6-hour floor (`MIN_WAITING_PERIOD`) is still enforced, so MANAGER never gets less than 6 hours, but the originally configured longer window is not preserved.

Impact

Both `DEFAULT_ADMIN` and `MANAGER` are trusted roles, and the 6h floor still applies. The issue is that the admin actions can implicitly reduce MANAGER's expected reaction time.

Code Snippet

<https://github.com/sherlock-audit/2026-04-lista-dao-may-7th-2026/blob/0358eaca561979b2b83b80a019882c722b434840/lista-new-contracts/src/lisaster/LisAsterDistributor.sol#L170-L174>

Tool Used

Manual Review

Recommendation

Disallow `changeWaitingPeriod` while a root is pending so the new value applies only to future stages:


```
function changeWaitingPeriod(uint256 newWaitingPeriod) external override
↳ onlyRole(DEFAULT_ADMIN_ROLE) {
+   require(pendingMerkleRoot == bytes32(0), "pending root exists");
   require(newWaitingPeriod >= MIN_WAITING_PERIOD, "waitingPeriod too short");
   waitingPeriod = newWaitingPeriod;
   emit WaitingPeriodUpdated(newWaitingPeriod);
}
```

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.